

職務経歴書

自己 PR

- 事業会社で勤務しており、現在は PM 業務をしながら FE (React、TypeScript)、BE (Go、Python、Rails) を触っております。

スキル

- 言語
 - JavaScript ・ TypeScript ・ Go ・ Python ・ Ruby
- フレームワーク等
 - React ・ Next.js ・ Rails
- RDB
 - MySQL
- クラウド
 - AWS
 - API Gateway ・ AppSync ・ Aurora ・ CloudWatch ・ Cognito ・ DynamoDB ・ ECR ・ ECS ・ EC2 ・ Fargate ・ IAM ・ Lambda ・ SES ・ SQS ・ S3
- SaaS/PaaS
 - AWS
 - Bitbucket ・ CircleCI ・ GitHub ・ GitHub Actions
- その他
 - Docker ・ GraphQL ・ RSpec

職務要約（現在）

- 勤務先
 - 事業会社

職務経歴詳細（現在）

- AI 利活用プロダクト（議事録・社内ツール）
 - 期間
 - 2024/04 ～ 現在
 - チーム編成
 - PM 1 人
 - 開発者 6 人
 - 言語・フレームワーク

- React・Next.js
 - Python・Django
 - 役割
 - PM・フルスタック
 - 実績
 - データ駆動による品質改善
 - 録音成功率が 70%と低迷していた課題に対し、GA（Google Analytics）を導入して失敗要因を可視化。ログ解析に基づき「技術的解決」と「ユーザー導線改善」を切り分け対策した結果、成功率を 87%まで向上
 - 録音の安定性向上のため、クライアント→サーバーのアップロードにマルチパートアップロードを実装。大きな音声データの分割送信と途中再開を可能にしたことで、ネットワーク途切れやタイムアウトによる失敗を大幅に削減し、録音成功率の改善に寄与
 - 新機能開発と利用量拡大
 - ディレクトリ管理機能や SFA 連携機能を PM 兼エンジニアとしてリード。リリース後、月間ドキュメント作成数を約 3 倍（3,750 件 → 10,947 件）に成長させた。
 - CS コストの削減
 - Dify を用いた AI チャットボットを導入し、問い合わせ対応を自動化。工数を抑えつつ迅速なユーザーフィードバックの収集体制を構築。
- レガシーシステムの内製化・移管プロジェクト
 - 期間
 - 2024/04 ~ 2024/09
 - チーム編成
 - PM 1 人
 - 開発者 2 人
 - 言語・フレームワーク
 - Vue.js
 - 役割
 - PM・フルスタック
 - プロジェクト概要
 - 外部ベンダー運用のインフラ・システムを自社内製チームへ移管
 - 実績
 - 円滑な技術承継
 - 複雑な AWS インフラ構成やソースコードを把握し、ベンダーからの引き継ぎをスケジュール通りに完結
 - オンボーディング改善
 - 特定のローカル開発環境で動作しない課題に対し、トラブルシューティングをまとめたドキュメントを整備。後続メンバーの参画コストを大幅に削減した
- BtoC 向けプラットフォームの新規開発
 - 期間
 - 2023/12 ~ 2024/09
 - チーム編成
 - PM 1 人
 - 開発者 10 人
 - 言語・フレームワーク
 - React・Next.js
 - Rails
 - 役割

- フロントエンド
- 実績
 - バグ・課題の集中消化
 - 約 100 件の未対応タスク・バグに対応。並行してコードのリファクタリングを積極的
に実施し、運用保守の負荷を軽減
 - ユーザビリティ改善の提案
 - 開発タスクをこなすだけでなく、ローディング画面の改善やバリデーション強化などを
事業部へ直接提案・実装し、UX 向上に寄与

職務要約（前職）

- 勤務先
 - 受託会社
- 役割
 - DX アプリケーション開発
- 業務内容
 - クライアントからの準委任契約のもと、労働時間を基にアプリの新規機能の開発、運用保守を担当して
おります。
 - クライアントとのコミュニケーションを密に取りながら、要件定義や設計、開発、運用までの一連の
流れを担当。

職務経歴詳細（前職）

- 不動産 DX プロダクト
 - 期間
 - 2023/4/1 ~ 2023/11/30
 - チーム編成
 - PM 1 人
 - 営業アシスタント 1 人
 - 開発者 2 人
 - 言語・フレームワーク
 - React
 - Go
 - Serverless Framework
 - インフラ
 - AWS
 - API Gateway・AppSync・Aurora・CloudWatch・Cognito・DynamoDB・ECR・
ECS・EC2・Fargate・IAM・Lambda・SES・SQS・S3
 - ツール
 - Bitbucket・Docker・GitHub・GitHub Actions・MySQL・Redmine
 - プロダクトが属する業界
 - 不動産業界
 - プロダクト概要
 - 2022/4/1 にアサインしていた時と同様
 - チームでの役割
 - バックエンド、フロントエンドの新規機能開発、レビューの担当
 - 手を動かすだけの開発だけではなく、画面設計・テーブル設計・API 設計・実装・テスト・リ
リース作業も担当

- 具体的なタスクは次の通り
 - Go + クリーンアーキテクチャ+ Lambda を用いた各種 API の作成
 - AppSync + Lambda + DynamoDB を使用した GraphQL 及びリゾルバーの作成
 - RDB や DynamoDB のテーブル設計およびインデックスの設定
 - IVR 認証機能の実装（バックエンド）
 - メディア 4u が提供している API と連携して実装
 - Controller 層、Repository 層の testing を使用したテーブル駆動のテスト
 - 通知の一括既読機能の実装
- 健康産業 DX プロダクト
 - 期間
 - 2022/11/1 ~ 2023/3/31
 - チーム編成
 - PM 2 人
 - 営業アシスタント 1 人
 - 開発者 2 人
 - 言語・フレームワーク
 - React・Next.js
 - Jest・Storybook
 - ツール
 - Bitbucket
 - プロダクトが属する業界
 - 健康産業界
 - チームでの役割
 - 新規機能開発、レビューの担当
 - 手を動かすだけの開発だけではなく、画面設計も担当
 - 具体的なタスクは次の通り
 - Figma をもとにフロント画面の作成
 - react-chartjs-2 を使用したチャートの作成
 - react-hook-form を使用したバリデーション
- 観光産業 DX プロダクト
 - 期間
 - 2022/9/1 ~ 2023/3/31
 - チーム編成
 - PM 2 人
 - 営業アシスタント 1 人
 - 開発者 2 人
 - 言語・フレームワーク
 - React・Next.js
 - Jest・Storybook
 - ツール
 - GitHub・GitHub Actions
 - プロダクトが属する業界
 - 観光産業界
 - チームでの役割
 - 新規機能開発、レビューの担当
 - 手を動かすだけの開発だけではなく、画面設計も担当
 - 具体的なタスクは次の通り

- Figma をもとにフロント画面の作成
 - react-pwa の導入
 - 対象デバイスがスマホのため、各画面のレスポンシブ化
 - QR コードの生成
- 不動産 DX プロダクト
 - 期間
 - 2022/4/1 ~ 2022/8/31
 - チーム編成
 - PM 1 人
 - 営業アシスタント 1 人
 - 開発者 2 人
 - 言語・フレームワーク
 - React
 - インフラ
 - AWS
 - API Gateway・AppSync・Aurora・CloudWatch・Cognito・DynamoDB・ECR・ECS・EC2・Fargate・IAM・Lambda・SES・SQS・S3
 - ツール
 - GitHub・GitHub Actions
 - プロダクトが属する業界
 - 不動産業界
 - プロダクト概要
 - 賃貸入居者と物件管理会社のやりとりをオンラインで完結するアプリケーション
 - 従来は、解約や騒音、水漏れなどの連絡は紙や電話を使用した方法が一般的で非常に非効率であった
 - 上記アプリケーションを導入することで、チャットベースでやりとりが行えるため、忙しい平日や夜中でも管理会社に連絡が可能になる
 - 他の機能として、掲示板機能、契約情報の確認、入居中サービス、各種申請など全てアプリケーション上から行える
 - チームの特徴・課題
 - 導入数が業界首位の入居者管理アプリケーションを開発元他社から引き継ぐことになり、一人目のリードフロントエンジニアとして参加
 - アサイン後、開発基盤を整備したことで、新機能実装や導入企業数が順調に増加し、東証グロース市場への上場に大きく貢献した
 - チームでの役割
 - 新規機能開発、レビューの担当
 - 手を動かすだけの開発だけではなく、画面設計・実装・テスト・リリース作業も担当
 - 具体的なタスクは次の通り
 - メールを配信できる数万人のユーザーをページング機能を用いて画面に表示
 - 表示されたユーザーの中から配信の有無を選択してチェックされているユーザーのみ id をバックエンドに送信する機能
 - 一括チェックボタンを使用して配信できる全ユーザーの id をバックエンドに送信する機能
 - 配信する内容を新規で作成、また過去に配信した内容を表示し編集でき、フロントで作成した内容をバックエンドに送信する機能
 - 複数の画面で使用されている右カラムの共通化
 - 7 画面で使用されている右カラムはコンポーネントを使用してレイアウトを共通化することでコード量を削減

- 画面の中のスクロール位置を取得して、一番下までスクロールした際にデータをリクエストする実装
 - API が未実装の際にフロント側でレイアウト、リクエストの確認ができるように AppSync で処理を記して mock 化
 - GraphQL ファイルにバックエンドと連携して構造体、フィールド名の記述
- チームの課題と自身が工夫したこと
 - ドキュメントの不足により開発スピードに影響が出ていたため全てドキュメントとして作成しチームに共有することで、チーム全体の業務効率化や生産性向上に大きく貢献した
 - 具体的な内容は次の 4 つ
 - バックエンドにリクエストを送る方法
 - mockAPI の作成手順
 - フロント側のデプロイ手順
- タクシー産業 DX プロダクト
 - 期間
 - 2022/1/5 ~ 2022/3/31
 - チーム編成
 - PM 1 人
 - 営業アシスタント 1 人
 - 開発者 7 人
 - 言語・フレームワーク
 - React・Next.js
 - Jest・Storybook
 - インフラ
 - AWS
 - ツール
 - GitHub・GitHub Actions
 - プロダクトが属する業界
 - タクシー業界
 - プロダクト概要
 - タクシーアプリと連動して使用する日本初のタクシーデリバリーに特化したサービス
 - ウェブ上で注文から決済まで完結できるサービス
 - チームの特徴・課題
 - React に知見ある人が少ない中での開発
 - 具体的なタスクは次の通り
 - 各画面のモジュールの実装
 - レイアウトが統一されているためコンポーネント化して各画面によって開閉のタイミング、表示する文言を変更できるように実装
 - カレンダーの実装
 - ライブラリーを使用せずにカレンダーを実装して配送日時を選択しバックエンドに送信できる機能
 - 2 つの画面で表示するカレンダーのコンポーネント化
 - 2 つの画面でレイアウトが異なるため、props を使用して style で変更する処理
 - 選択された値をバックエンドに送る処理はカスタムフックを作成して共通化
 - ライブラリー(swiper)を使用して画像のスライド機能の実装
 - スライド機能のコンポーネント化
 - 画面によって表示するレイアウトのサイズ、スライドのアクションを変更できるように実装
 - 作成した画面のユニットテスト

- storybook を使用して UI の構築